

Structure de HCL



HCL

Anatomie d'un main.tf

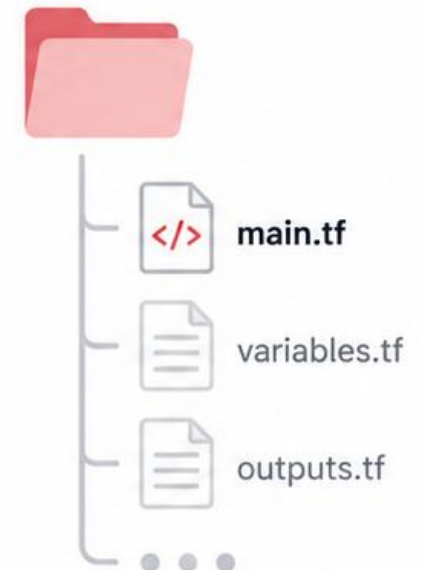
Le fichier main.tf est le point d'entrée classique d'un projet Terraform.

Structure typique

- 1 Bloc terraform
- 2 Bloc provider
- 3 Variables
- 4 Locals
- 5 Resources
- 6 Outputs

Exemple de main.tf

```
terraform {  
  required_version = ">= 1.5.0"  
}  
  
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_instance" "web" {  
  ami          = "ami-12345678"  
  instance_type = "t2.micro"  
}  
  
output "instance_id" {  
  value = aws_instance.web.id  
}
```



Objectif : comprendre où se placent les blocs dans un fichier Terraform.



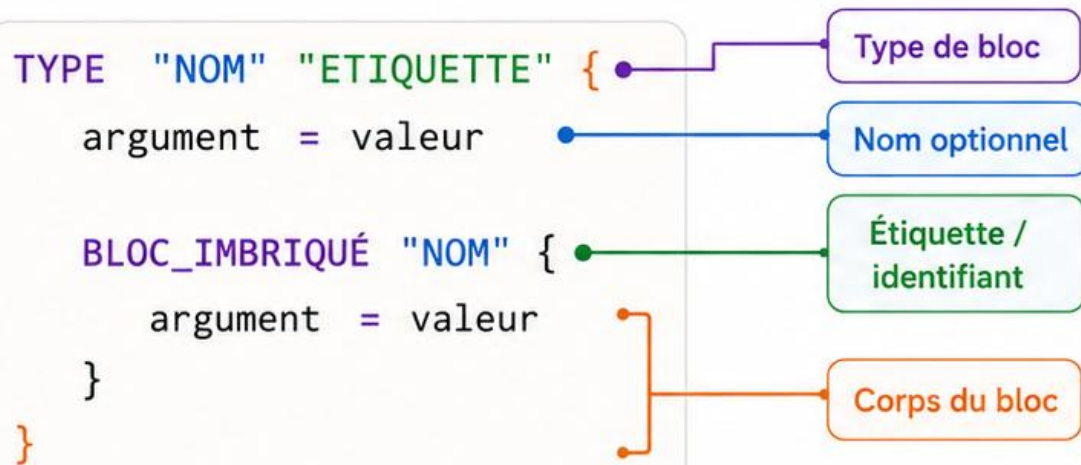
HCL

Anatomie d'un bloc

Un bloc est la structure de base en HCL. Il définit un élément comme terraform, provider, resource, module ou output.



Structure générale

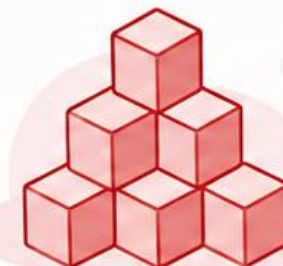


Un bloc commence par un type, peut avoir un nom et une étiquette, puis contient des arguments ou d'autres blocs imbriqués entre accolades.



Exemple

```
resource "aws_instance" "web" {  
  ami           = "ami-12345678"  
  instance_type = "t2.micro"  
  
  tags = {  
    Name = "web-server"  
  }  
}
```



Objectif : comprendre la structure d'un bloc HCL et le rôle de ses éléments.



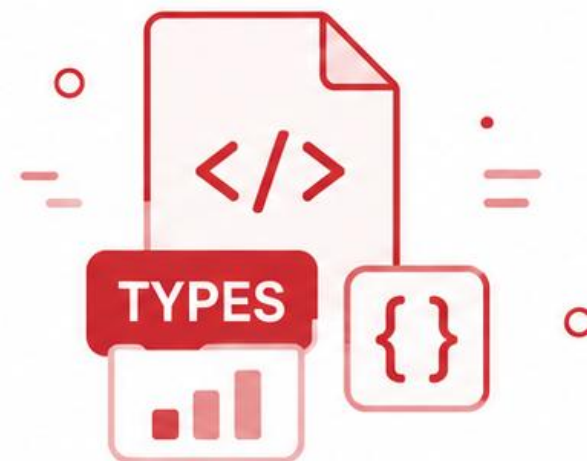
HCL

Types de données

HCL prend en charge plusieurs types de données pour représenter les valeurs d'une configuration.

Exemples de types

Type	Exemple
string	"hello"
number	42
boolean	true
list	["a", "b", "c"]
map	{ key = "value" }
set	toset(["a", "b"])
object	{ name = "web", port = 80 }
tuple	["web", 80, true]



HCL est **fortement typé** :
chaque valeur a un type.



Objectif : reconnaître les principaux types de données utilisés dans HCL.



HCL

Références

Les références permettent de relier les éléments d'une configuration Terraform entre eux.



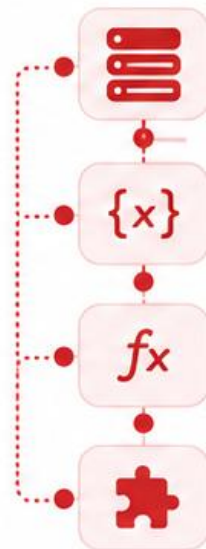
Types de références

-  Ressources → `aws_instance.web.id`
-  Variables → `var.region`
-  Locals → `local.name`
-  Modules → `module.vpc.public_subnet_id`



Exemple

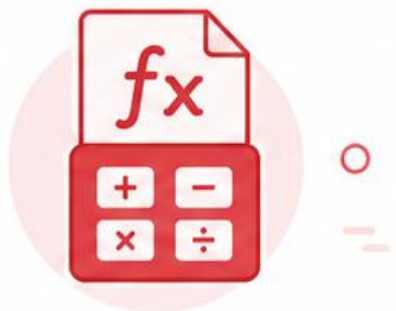
```
resource "aws_instance" "web" {  
  ami           = var.ami_id  
  instance_type = "t2.micro"  
  subnet_id     = module.vpc.public_subnet_id  
  
  tags = {  
    Name       = local.name  
    Environment = var.env  
  }  
}
```



Les références créent des dépendances automatiques entre ressources.



Objectif : comprendre comment HCL relie les ressources, variables, locals et modules.



HCL

Expressions

Les expressions HCL permettent de calculer des valeurs et de rendre la configuration dynamique.



Fonctionnalités



Opérateurs arithmétiques → +, -, *, /



Comparaison → ==, !=, <, >



Logique → &&, ||, !



Fonctions intégrées → length(), upper(), cidrsubnet()



Conditionnelle → condition ? valeur_true : valeur_false



Exemples

```
local.x = var.a * 2 + 1
local.is_prod = var.env == "prod" ? 1 : 0
local.name_prefix = upper(var.app_name)

resource "aws_instance" "web" {
  count          = local.is_prod ? 2 : 1
  ami            = var.ami_id
  instance_type = "t2.micro"

  tags = {
    Name = "${local.name_prefix}-${count.index}"
  }
}
```



Les expressions
rendent la
configuration
dynamique et
réutilisable.



Objectif : comprendre comment HCL calcule et adapte les valeurs.